

HOJA DE REFERENCIA MIPS 32

Arquitectura del Computador.

Penélope Robledo.

C.I:32482938.

Sección 1.

CONCEPTOS GENERALES

Estructura de un Archivo Ensamblador: Todo programa se divide en dos secciones principales mediante directivas:

- **.data:** Aquí se declaran los datos. Físicamente se alojan las variables en RAM, mensajes, cadenas de texto (**.asciiz**) y arreglos.
- **.text:** Aquí reside el código. Se escribe la lógica del programa, las instrucciones a ejecutar directamente por el procesador, y la etiqueta de inicio **main**:

Ejemplo de una Sintaxis Estructural Inicial:

```
.data
    numero: .word 25          # Variable entera comun
    texto:   .asciiz "Hola"   # Cadena de texto (String)

.text
main:
    lw $t0, numero           # Inicia el codigo cargando un valor
```

Organización de una Línea de Instrucción MIPS:

- **OPCODE / INSTRUCCIÓN:** Especifica la operación a realizar (ej: 'add').
- **REGISTRO DESTINO:** El registro donde se almacenará el resultado.
- **OPERANDOS:** Los elementos de origen desde donde saldrá la información.

Principales Registros (De un total de 32):

- **\$t0-\$t9:** Registros temporales para almacenar operaciones o contadores de corto plazo.
- **\$a0-\$a3:** Paso de argumentos requeridos para rutinas y el sistema.
- **\$v0-\$v1:** Valores retornados por funciones y códigos que solicitan un **syscall**.
- **\$ra:** Dirección de retorno requerida al utilizar un salto tipo **jal**.

A. Operaciones Aritméticas

1. ADD (Suma de Registros)

Instrucción: Suma los valores contenidos en dos registros de origen y almacena el resultado en el registro destino.

```
add $t0, $t1, $t2    # Sumar registros
```

Explicación: Resulta útil cuando los dos números a sumar ya están cargados dentro de registros temporales.

2. ADDI (Suma con Inmediato)

Instrucción: Suma un registro de origen con un valor constante (número directo) y guarda el resultado.

```
addi $s0, $s1, 100 # Sumar constante
```

Explicación: Ideal para sumar un número específico directamente o incrementar el contador dentro un bucle ($i++$).

3. SUB (Resta de Registros)

Instrucción: Resta el contenido del segundo registro de origen al primero y guarda la diferencia.

```
sub $t0, $t1, $t2 # Restar registros
```

Explicación: Ejecuta la sencilla operación matemática de $\$t1$ menos $\$t2$.

4. SUBI (Resta con Inmediato)

Instrucción: Resta un valor numérico constante al registro de origen temporal.

```
subi $s0, $s1, 50 # Restar constante numerica
```

Explicación: Estrictamente hablando es una pseudoinstrucción. El procesador ahorra trabajo escribiéndola y se encarga de cambiarla internamente a una suma negativa normal (`addi $s0, $s1, -50`).

B. Transferencia de Datos

5. LI (Cargar Inmediato)

Instrucción: Carga un número directo de 32 bits a un registro específico.

```
li $t0, 50 # Cargar numero directo
```

Explicación: Coloca el número 50 directamente en el registro $\$t0$. Es la forma más fácil y común de inicializar una variable numérica.

6. LA (Load Address)

Instrucción: Carga la dirección de memoria de una etiqueta creada en la sección `.data`.

```
la $a0, mensaje # Cargar la direccion del texto
```

Explicación: Es obligatoria para indicarle al sistema la ubicación de un texto (String) al momento de imprimirlo, o para leer los elementos de un arreglo grande.

7. MOVE (Copiar)

Instrucción: Copia el valor actual de un registro de origen hacia un registro de destino.

```
move $t0, $t1      # Copiar contenido de $t1 en $t0
```

Explicación: Resulta útil para transferir variables o resultados guardados hacia el registro de argumentos \$a0, para luego mostrarlos por pantalla.

8. LW (Load Word)

Instrucción: Carga una Palabra completa (32 bits o 4 bytes) desde una dirección en la memoria RAM hacia un registro.

```
lw $t1, 0($s0)     # Traer los bits desde memoria a t1
```

Explicación: Es la instrucción principal para extraer variables numéricas que están guardadas en la memoria de la computadora y poder trabajar con ellas.

9. SW (Store Word)

Instrucción: Guarda el contenido completo (32 bits) de un registro en una dirección específica de la memoria RAM.

```
sw $t1, 4($s0)     # Guardar en memoria (Desplazamiento 4)
```

Explicación: Hace exactamente lo contrario a LW. Se utiliza para guardar resultados finales o modificar los datos que residen en la RAM.

10. LH (Load Halfword)

Instrucción: Carga solo la mitad (16 bits) de un dato desde la memoria hacia un registro.

```
lh $t2, 0($s0)     # Cargar fraccion de 16 bits
```

Explicación: Ayuda a leer números más pequeños para ahorrar recursos en el procesador. Al cargar el dato, mantiene correctamente el signo (positivo o negativo).

11. SH (Store Halfword)

Instrucción: Guarda únicamente la mitad inferior (16 bits) del valor de un registro en la memoria RAM.

```
sh $t2, 0($s1)     # Guardar solamente 16 bits
```

Explicación: Es ideal para optimizar el uso de la memoria cuando se trabaja con arreglos que almacenan números pequeños preestablecidos.

12. LB (Load Byte)

Instrucción: Carga un solo byte (8 bits) desde la memoria hacia un registro.

```
lb $t3, 0($s0)     # Cargar un caracter individual
```

Explicación: Es obligatoria y muy utilizada para leer, letra por letra, los caracteres individuales (código ASCII) que forman un texto o String.

13. SB (Store Byte)

Instrucción: Guarda únicamente los últimos 8 bits (1 byte) de un registro en la memoria RAM.

```
1 sb $t3, 1($s1)      # Escribir 1 simple byte
```

Explicación: Permite modificar caracteres individuales dentro de un texto, o guardar variables muy pequeñas (como valores booleanos) sin afectar el resto de la memoria.

14. LUI (Load Upper Immediate)

Instrucción: Carga un número constante en la mitad superior de un registro y rellena la mitad inferior con ceros.

```
1 lui $t0, 0xABCD     # Cargar en la parte alta (Upper)
```

Explicación: Sirve para construir manualmente direcciones de memoria muy grandes que superan el límite de tamaño de una instrucción normal.

C. Operaciones Lógicas

15. AND (Y Lógico)

Instrucción: Realiza una operación lógica "Y" bit a bit. El resultado será 1 solo si ambos bits comparados son 1.

```
1 and $t0, $t1, $t2   # Operacion AND entre registros
```

Explicación: Se usa para seleccionar fragmentos de un dato. Al aplicar la operación, cualquier bit que choque contra un 0 se apaga automáticamente, mientras que los bits que chocan contra un 1 mantienen su valor original.

16. ANDI (Y Lógico Inmediato)

Instrucción: Realiza la misma operación lógica "Y", pero compara un registro contra un número constante (inmediato).

```
1 andi $t0, $t1, 0xFF # Operacion AND con un numero directo
```

Explicación: Se usa para ignorar o limpiar ciertos bits utilizando un valor límite que ya se conoce desde el inicio de la programación.

17. OR (O Lógico)

Instrucción: Realiza una operación lógica 'O' bit a bit. El resultado será 1 si al menos uno de los bits comparados es 1.

```
1 or $t0, $t1, $t2    # Operacion OR entre registros
```

Explicación: Es útil para encender ciertos bits de un registro de forma intencional, sin modificar los bits restantes del parámetro.

18. ORI (O Lógico Inmediato)

Instrucción: Realiza la operación lógica 'O', pero utilizando un número constante (inmediato) directo como límite.

```
ori $t0, $t1, 0x0F # Operación OR con una base constante
```

Explicación: Funciona perfectamente para forzar el encendido de ciertos bits comparándolos directamente con un valor manual.

19. NOR (O Negado)

Instrucción: Realiza una operación 'O Negado'. Invierte los bits de los registros de entrada y da el resultado opuesto.

```
nor $t0, $t1, $zero # Invierte todos los bits de $t1
```

Explicación: Este método simplifica la programación, ya que se utiliza para invertir completamente los bits de un registro (simulando una operación lógica NOT tradicional).

20. SRL (Shift Right Logical)

Instrucción: Desplaza los bits de un registro hacia la derecha, rellenando con ceros los espacios que quedan vacíos a la izquierda.

```
srl $t0, $t1, 2 # Desplazar 2 posiciones a la derecha
```

Explicación: Matemáticamente, desplazar los bits a la derecha equivale a dividir el número entre 2 por cada posición movida (mover 2 posiciones divide el número entre 4).

D. Saltos y Condicionales (Comparadores)

21. BEQ (Igual) / BNE (Diferente)

Instrucción: Determina si dos registros son iguales (beq) o diferentes (bne). Si se cumple la condición, el programa salta a la etiqueta indicada.

```
beq $t0, $t1, Etiqueta # Si (t0 == t1) saltar a Etiqueta
bne $t0, $t1, Etiqueta # Si (t0 != t1) saltar a Etiqueta
```

Explicación: Resultan indispensables para implementar condicionales del tipo if o if-else, bifurcando la ejecución del programa según el resultado de la comparación de dos registros.

22. BLT y BLE (Menor / Menor o igual)

Instrucción: Salta a una etiqueta si el primer registro es estrictamente menor (<) o menor o igual (<=) que el segundo registro.

```
blt $t0, $t1, Etiqu # Si (t0 < t1) saltar a Etiqu
ble $t0, $t1, Etiqu # Si (t0 <= t1) saltar a Etiqu
```

Explicación: Permiten evaluar condiciones de límites inferiores, siendo ideales para controlar estructuras repetitivas (bucles) o para validar rangos numéricos acotados.

23. BGT y BGE (Mayor / Mayor o igual)

Instrucción: Salta a una etiqueta si el primer registro es estrictamente mayor (>) o mayor o igual (>=) que el segundo registro.

```
1 bgt $t0, $t1, Etiqu # Si (t0 > t1) saltar a Etiqu
2 bge $t0, $t1, Etiqu # Si (t0 >= t1) saltar a Etiqu
```

Explicación: Funcionan de manera inversa a las anteriores, resultando muy útiles para verificar que un contador no supere un tope máximo preestablecido o para validar el ingreso de datos superiores a un umbral.

24. SLT / SLTI (Set Less Than)

Instrucción: Compara dos valores. Si el primer registro es menor que el segundo, guarda un 1 en el registro destino; de lo contrario, guarda un 0.

```
1 slt $t0, $s1, $s2 # Guarda un 1 si $s1 < $s2
2 slti $t0, $s1, 10 # Guarda un 1 si $s1 < 10 (con constante)
```

Explicación: Es una herramienta muy útil para evaluar condiciones y guardar un resultado de verdadero (1) o falso (0) dentro del sistema, de manera similar a la programación estructurada clásica.

E. Saltos Incondicionales (Directos)

25. J (Jump)

Instrucción: Interrumpe el flujo normal del programa y fuerza un salto directo hacia una etiqueta específica.

```
1 j Fin_bucle # Salta directamente a la etiqueta Fin_bucle
```

Explicación: Desvía la ejecución del código sin evaluar ninguna condición previa y sin importar lo que estuviera haciendo el procesador.

26. JAL (Jump And Link)

Instrucción: Salta a una etiqueta (como una función) y al mismo tiempo guarda la dirección actual en el registro de retorno (\$ra).

```
1 jal FuncionA # Llama a la funcion y guarda la direccion de
    retorno
```

Explicación: Permite que el procesador recuerde exactamente de dónde vino antes de saltar. Es indispensable para crear funciones o subrutinas, ya que asegura que el programa sepa a dónde debe regresar al terminar.

27. JR (Jump Register)

Instrucción: Salta a la dirección de memoria que está guardada dentro de un registro, que casi siempre es el registro \$ra.

```
jr $ra           # Regresar a quien llamo a esta funcion
```

Explicación: Es la instrucción complementaria de jal. Se utiliza al final de una función para forzar al programa a regresar al punto exacto donde la función fue solicitada originalmente.

F. Llamadas al Sistema (SYSCALL)

Funciones del Sistema (Input y Consola)

La instrucción `syscall` detiene momentáneamente el programa y le indica al sistema operativo o simulador que ejecute una acción en la consola. El sistema sabrá exactamente qué acción realizar dependiendo del número que hayamos guardado previamente en el registro `$v0`.

La lista principal de acciones que el sistema puede ejecutar es la siguiente:

- `li $v0, 1`: **Imprimir un Entero** (Muestra en pantalla el número que se encuentra en el registro `$a0`).
- `li $v0, 4`: **Imprimir un Texto / String** (Requiere que la dirección del texto esté guardada previamente en el registro `$a0` usando `la`).
- `li $v0, 5`: **Leer un Entero** (Pausa el programa y espera a que el usuario escriba un número. El valor ingresado se guardará en el registro `$v0`).
- `li $v0, 8`: **Leer un String** (Permite al usuario escribir un texto y lo guarda directamente en la memoria RAM, en una dirección asignada).
- `li $v0, 10`: **Finalizar el Programa** (Opción directa para cerrar y salir del programa de forma segura).

Ejemplo General en Programa: Imprimir un Texto

La metodología en el código sigue siempre el mismo orden: primero se le indica al registro `$v0` el código de la acción, luego se preparan los argumentos necesarios en `$a0`, y finalmente se llama a `syscall` para ejecutar:

```
.data
    mensaje1: .asciiz "Demostracion Hola Mundo\n"

.text
main:
    # ---- PASO A NIVEL DE IMPRESION ----
    li $v0, 4          # El codigo 4 indica: Imprimir texto en
                        # consola
    la $a0, mensaje1   # Pasamos la ubicacion del texto al
                        # registro $a0
    syscall            # El sistema ejecuta la orden e imprime en
                        # pantalla

    # ---- PASO FINAL PARA CERRAR EL SISTEMA ----
    li $v0, 10         # El codigo 10 indica: Finalizar programa
                        # de forma segura
    syscall            # El sistema ejecuta la orden y cierra el
                        # programa
```

G. Lógica Básica de Ciclos (FOR y WHILE)

Para que sea más fácil de entender, la forma en que MIPS construye sus ciclos iterativos se puede comparar directamente con la sintaxis clásica de lenguajes como C/C++.

Ciclo Básico WHILE

En MIPS32, los ciclos WHILE se construyen utilizando una condición de salida **inversa**. Esto significa que, en lugar de evaluar cuándo continuar, se evalúa la condición para saber cuándo **romper** o escapar del ciclo. Si esta condición se cumple, el programa salta hacia afuera y termina el bucle. Para repetir el proceso, al final del bloque de código se utiliza un salto incondicional (j) que obliga al programa a regresar al inicio del ciclo para evaluarse de nuevo.

Interpretación equivalente (Sintaxis C):

```
int i = 0;
while (i < 10) {           // La lógica es repetir mientras se
    // cumple (i < 10)
    // Bloque de instrucciones internas
    i++;
}
```

Equivalencia construida sobre simulador MIPS32:

```
li $t0, 0                  # Se inicializa la variable contador
: i = 0

Ciclo_While:
    bge $t0, 10, Fin_While # Condicion INVERSA para salir. Si "i
                           # >= 10", el programa salta a Fin_While y escapa del ciclo.

    # ... Bloque de instrucciones MIPS a repetir ...

    addi $t0, $t0, 1        # Se incrementa el contador: i++
    j Ciclo_While           # Salto incondicional para regresar
                           # al inicio del ciclo

Fin_While:
    # El programa continua por aqui una vez que el ciclo se rompe
    .
```

Ciclo Básico FOR

El ciclo FOR requiere que se definan de antemano dos variables principales: un registro que servirá como contador y otro registro que servirá como el límite máximo. A diferencia de otros bucles, la condición de salida suele utilizar un comparador de igualdad (**beq**) para que el sistema reconozca el momento exacto en que el contador alcanza el tope límite y, de esa forma, detener el ciclo rompiéndolo adecuadamente.

Interpretación equivalente (Sintaxis C):

```
for (int i = 0; i != 5; i++) {  
    // Bloque de instrucciones a repetir  
}
```

Equivalencia construida sobre simulador MIPS32:

```
li $t0, 0           # Se define el contador: i = 0  
li $t1, 5           # Se asigna el limite maximo: tope =  
                    5  
  
Ciclo_For:  
    beq $t0, $t1, Fin_For    # Condicion de salida: Si el contador  
                             alcanza el limite (i == 5), salta y rompe el ciclo.  
  
    # ... Instrucciones y funciones a repetir ...  
  
    addi $t0, $t0, 1         # Incremento del contador en cada  
                             repeticion: i++  
    j Ciclo_For             # Retorna obligatoriamente al inicio  
                             del ciclo FOR  
  
Fin_For:  
    # Fin de la repeticion y continuacion del programa.
```